```
.
├── companyms
│   ├── docker-compose.yml
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   ├── src
│   └── target
├── configserver
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   ├── src
│   └── target
├── gateway
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   ├── src
│   └── target
├── jobms
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   ├── src
│   └── target
├── project-structure.txt
├── reviewms
│   ├── HELP.md
│   ├── mvnw
│   ├── mvnw.cmd
│   ├── pom.xml
│   ├── src
│   └── target
└── service-reg
    ├── HELP.md
    ├── mvnw
    ├── mvnw.cmd
    ├── pom.xml
    ├── src
    └── target
```

Figure 6: Folder structure of a microservices-based job portal

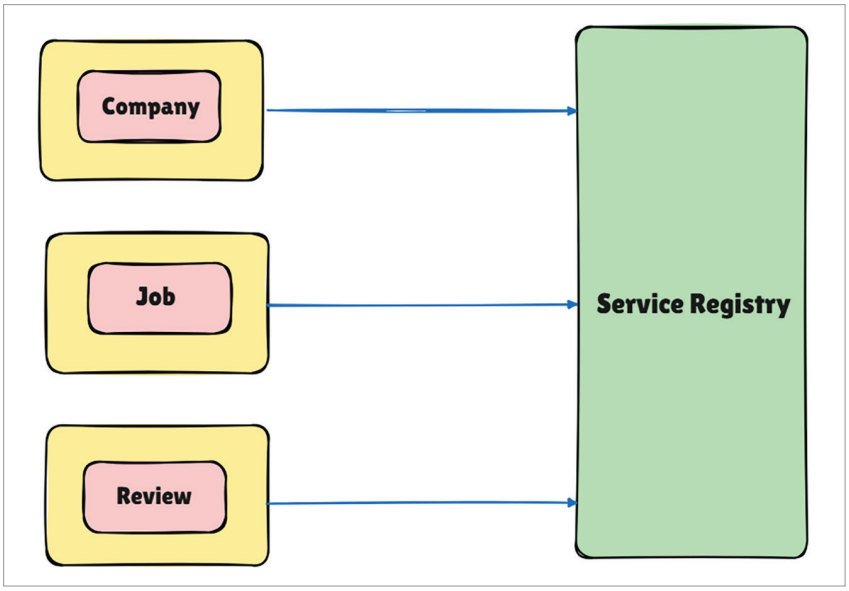

Figure 7: Service registry

systems where network calls may be expensive or slow. DTOs are often used in combination with other patterns, such as the DAO (Data Access Object) pattern, and are particularly useful in web services and APIs where efficient data transfer is crucial.

Service registry

A service registry is a centralised database or repository that maintains information about available services in a distributed system or microservices architecture. It acts as a directory where services register themselves upon startup, providing metadata such as the service name, network location, and status. The registry allows other services to discover and connect to the registered services dynamically without hardcoding their locations. This is especially useful in environments where services may scale up or down, and their network addresses change frequently. Service registries are often used in conjunction with service discovery mechanisms, which help clients find and access services by querying the registry. Tools like Eureka are commonly used to implement service registries as demonstrated in Figure 7, ensuring high availability, fault tolerance, and scalability in microservices-based architectures.

Fault tolerance

Fault tolerance refers to the ability of a system to continue functioning correctly even in the presence of failures. A fault-tolerant system is specifically designed to handle unexpected disruptions, such as hardware failures, network issues, or software bugs, without compromising the overall user experience or service reliability. To achieve fault tolerance, systems often employ redundancy, graceful degradation, and various error-handling mechanisms, such as retries, timeouts, and fallback procedures. For instance, critical components may be replicated across multiple servers or regions to ensure that if one server fails, another can take over seamlessly, minimising service interruptions.

One key strategy for maintaining fault tolerance, particularly in microservices architectures, is the implementation of circuit breakers. A circuit breaker, such as the companyBreaker instance used in this system, monitors service interactions, specifically tracking failure rates and slow call rates. The circuit breaker ensures that if the failure or slow call